Part A — Your Work

1. What part of the project did you personally work on?

The project I personally work on the Inheritance which is the Equipment and the RentalRecord classes and the implementation of both classes. The Equipment class stores and manages information of every equipment items. The RentalRecord class is the receipt/transaction for renting the items.

2. What classes, functions, or files did you contribute to?

The classes I work on is:

- Equipment.h / Equipment.cpp
- RentalRecord.h / RentalRecord.cpp

The Equipment.h / Equipment.cpp Functions I work on is:

- Equipment::Equipment()
- displayDetails(),useItem(), typeTag()
- rent(), returnItem()
- bool operator==(), operator<<
- getId(), getName(), getBasePrice(), IsAvailable(), getDescription()

RentalRecord.h / RentalRecord.cpp the functions were

- RentalRecord::RentalRecord()
- calculateFee(), isOverdue(),
- operator+()

- displayRecord()
- getRecordId()
- getCustomerName()
- getRentedItem()
- getRentalDay()
- getRentalDuration()
- getTotalFee()
- setTotalFee()
- setRentedItem()

the equipment is the base inventory/item system and the rental record is transaction and fee system.

3. What decisions did you help make?

I crate an equipment class for all rentable items because you don't have rewrite the same code in weapon, armor, and potion. You have all the shared data and the behavior in one parent class.

Part B — OOP Understanding

4. Why was a class-based design better than writing everything in one file?

A class-based design better than writing everything in one file because to be better organization. Each classes had a difference responsibility and without the classes the problem would be harder to read and manage.

5. How did your team use encapsulation?

In my part of code, the equipment fields are protected and not public because to support inheritance while still maintain encapsulation. Using protected allows derived classes to access

and reuse these fields while preventing unrelated outside code from modifying them directly. This design improves data protection, supports polymorphism and inheritance, and keeps the project more organized and maintainable.

6. What is the purpose of inheritance in your project?

The Equipment base class exists because all rentable items in the project share common data and behaviors. Weapons, armor, and potions all need properties such as an ID, name, rental price, availability status, and description. Instead of duplicating this code in every class, the base class captures the shared structure once and allows derived classes to inherit it.

7. How did polymorphism work in your system?

Polymorphism in our system worked through the `Equipment` base class and its virtual functions. Classes such as `Weapon`, `Armor`, and `Potion` inherited from `Equipment` and overrode functions like `displayDetails()`, `useItem()`, and `typeTag()` with their own specialized behavior also help from Jodestee.

8. Why was the base class useful?

The Equipment base class was useful because it centralized the shared data and behavior for all rentable items into one reusable structure. Instead of duplicating common variables and functions in every item class, derived classes such as Weapon, Armor, and Potion inherited shared features like IDs, names, prices, availability, descriptions, and common behaviors.

9. Which operator was overloaded and why?

Our project overloaded three operators: ==, <<, and +.

The == operator was overloaded in the Equipment class to compare equipment objects by their IDs, making item comparisons simpler and more readable. The << operator was overloaded to allow equipment objects to be printed directly using cout, which improved output formatting and

debugging. The + operator was overloaded in the RentalRecord class to combine rental fee totals for revenue calculations and daily reports. Operator overloading made the code more intuitive, reduced repetitive logic, and demonstrated object-oriented programming concepts required by the project.

10. How was the template useful in your program?

There is no template in my part of the code.

Part C — Technical Learning

11. How did file handling work in your project?

I didn’t write any of file handling work in my part of project.

12. What role did stringstream play, if any?

Prompt: It splits each CSV line into fields with std::getline(ss, token, ','). Without it, parsing would be a manual mess of finding comma positions.

Your answer here:

13. What exception cases did your project handle?

Prompt: Three custom (InvalidItemID, NegativeGoldException, FileAccessException) plus two standard (invalid_argument, out_of_range). Pick one and explain when it gets thrown.

Your answer here:

14. What debugging problem did you face and how did you solve it?

The problems I have faced was overloading operators, the way I solve it was looking at professor slide and using AI fix any problems I have.

15. What did you learn about organizing a multi-file project?

You can use classes and put them into main.cpp for nice organizing.

Part D — AI Use and Academic Integrity

16. How did you use AI tools during the project?

I used AI tools for the overloading operation help me understand what needs to be done.

17. What was useful about the AI support?

AI support help me understand coding of overloading examples from the lesson slides.

18. Did the AI ever suggest something wrong or unhelpful?

Yes, when I ask something different and not showing what I want of helping understand what is wrong.

19. How did you verify that the code or explanation was correct?

Lesson slides from the professor.

20. How did AI support your learning instead of replacing your thinking?

AI support my learning by asking different question that I have to come up with.

Part E — Reflection

21. What was the hardest part of this project?

The hardest part of this project was understanding how the code works and what went wrong of my code.

22. What was the most valuable thing you learned?

The most valuable thing I learn was create a classes and implementation.

23. What would you improve if you had more time?

I would improve if I had more time is the overloading operators because I still don’t how it work or understand it.

24. What are you most proud of in this project?

Focusing on the project at my end and learning new things every day.

25. How did this capstone help you understand OOP more deeply?

Prompt: Before vs after. Before, you knew the words. After, you've used the concepts on a real project where they earned their place. Pick one or two concepts and say HOW your understanding changed.

Your answer here: